1. 课程目标

- 理解闭包的概念及其实现条件。
- 掌握装饰器的核心思想及其应用场景。
- 学会使用闭包和装饰器扩展函数功能。

---

2. 闭包函数

- **闭包的定义**：
  - 闭包是一个内部函数，它引用了外部函数的变量，并且外部函数返回了内部函数。
- **闭包的形成条件**：
  1. **嵌套函数**：在一个函数内部定义另一个函数。
  2. **内部函数引用外部函数的变量**：内部函数使用了外部函数的变量或参数。
  3. **外部函数返回内部函数**：外部函数将内部函数作为返回值返回。
- **示例代码**：

python 复制

```python
def outer(x, y):
    def inner():
        print(x + y)
    return inner

func = outer(1, 2)
func()  # 输出 3
```

---

3. 装饰器

- **装饰器的定义**：
  - 装饰器是一个函数，它接受一个函数作为参数，并返回一个新的函数，用于扩展原函数的功能。
- **装饰器的核心思想**：
  - **不修改被装饰对象的源代码。**
  - **不修改被装饰对象的调用方式。**

- 示例代码：

```python
def run_time(func):
    def wrapper(*args, **kwargs):
        start_time = datetime.now()
        print('开始时间%s' % start_time)
        result = func(*args, **kwargs)
        end_time = datetime.now()
        print('结束时间%s' % end_time)
        time1 = end_time - start_time
        print('花费时间%s' % time1)
        return result
    return wrapper

@run_time
def for1(n):
    sum1 = 0
    for i in range(1, n + 1):
        sum1 += i
    print(sum1)

for1(9000000)
```

---

4. 装饰器的应用场景

- 日志记录：

```python
def log(func):
    def wrapper(*args, **kwargs):
        print(f"调用函数：{func.__name__}")
        result = func(*args, **kwargs)
        print(f"函数 {func.__name__} 执行完毕")
        return result
    return wrapper

@log
def add(a, b):
    return a + b

print(add(1, 2))
```

- 性能测试：

```python
@run_time
def shopping():
    for i in range(1111111):
        pass
    print('我是shopping')

shopping()
```

---

5. 闭包与装饰器的关系

- **装饰器是闭包的一种应用**：
  - 装饰器本质上是一个闭包函数，它通过闭包捕获被装饰的函数，并扩展其功能。
- **示例代码**：

```python
def decorate(func):
    def new_func(name):
        print('我是装饰函数前面的代码')
        func(name)
        print('我是装饰函数后面的代码')
    return new_func

@decorate
def run(name):
    print('我是%s' % name)

run('大海')
```

---

6. 总结

- **闭包**：通过嵌套函数和变量引用实现状态保存。
- **装饰器**：在不修改原函数代码和调用方式的情况下，扩展函数功能。
- **应用场景**：日志记录、性能测试、权限验证等。